

Notes from terraform video: how I understand terraform

There are difference IaC tools types:

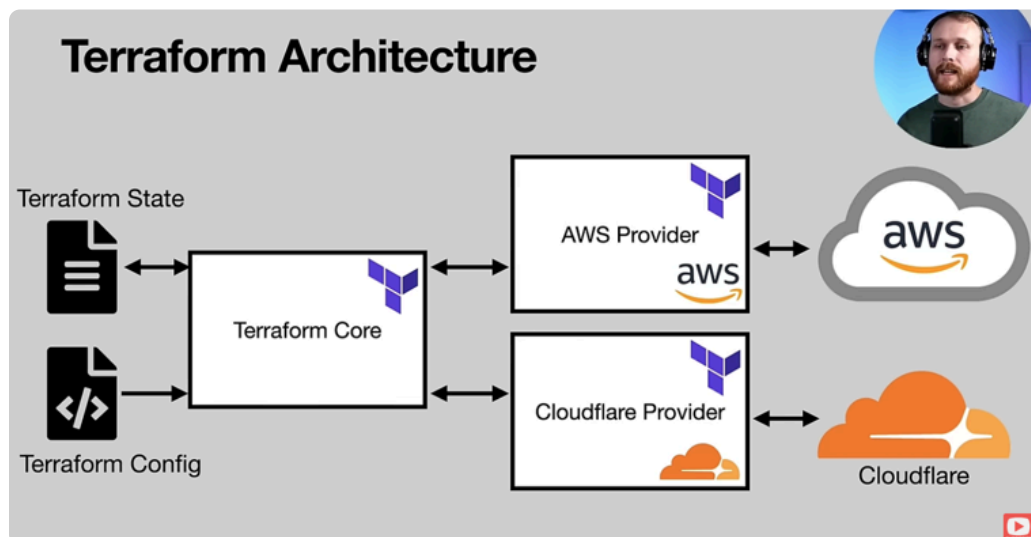
1. ad hoc scripts
2. configuration management tools
3. server templating tools
4. orchestration tools
5. **provisioning tools**

These tools can be used in declarative or imperative ways.

- **Declarative:** means you describe **what you want**, and the system figures out **how to do it**.
(define then end state of what you want: I want 3 S3 buckets, 2 load balances etc; then the tool manages what API calls need to be made and how to actually make that happen)
- **Imperative:** means you give **step-by-step instructions** on **how to get the result**.

Terraform falls into a provisioning tool and is primarily on the declarative side (you specify the end tool you want your infrastructure to take and the program will figure out how to get there)

Terraform architecture



The terraform core will take your current terraform file configurations and terraform state; and it will figure out how to interact with the cloud providers API to create the resource. The provides tell terraform how to map a specific configuration on the current state of the AWS API.

NOTE: Terraform is can run so many different provides, it also can use providers that rant strictly coded to be compatible with the software because it is open source. Can run third providers throw third parties aswell (more off a good note for that trading project you want to do).

What happens when different terraform commands are ran:

Terraform init = when this is ran it goes off and downloads the code needed to run the provider that was specified in the [main.tf](#) file in the working directory.

The seconded thing it does is that if you have used any modules (modules are a way of bundling up terraform code so it is reusable) so if you are using any of those it will also get them and down load them into the working directory as well.

State file = This Json file contain information about every single resource and data object that we have deployed using terraform. **This will contain sensitive information.** Whenever you make a terraform project you **HAVE TOO git ignore this** because the repo is at risk of being viewed whether it is public or private. As you can see bellow there is the API key and other sensitive data.

```

{
  "version": 4,
  "terraform_version": "1.15.8",
  "serial": 111,
  "lineage": "d1423b4c-efe0-13bb-5c37-32ebee6e5f06",
  "outputs": {
    "api_base_url": {
      "value": "https://np6w972j8k.execute-api.eu-north-1.amazonaws.com/dev",
      "type": "string"
    },
    "api_key_id": {
      "value": "ynr7pydtw1",
      "type": "string"
    },
    "dynamodb_table_name": {
      "value": "umar-coldchain-database-terraform",
      "type": "string"
    },
    "excursion_handler_lambda_name": {
      "value": "umar-coldchain-excursion-handler-terraform",
      "type": "string"
    },
    "excursion_handler_role_name": {
      "value": "umar-coldchain-excursion-handler-role-terraform",
      "type": "string"
    },
    "link_dynamodb_lambda_name": {
      "value": "umar-coldchain-LinkDynamoDB-terraform",
      "type": "string"
    },
    "link_dynamodb_role_name": {
      "value": "umar-coldchain-LinkDynamoDB-role-terraform",
      "type": "string"
    },
    "postman_readings_url": {
      "value": "https://np6w972j8k.execute-api.eu-north-1.amazonaws.com/dev/branches/BRANCH-001/readings",
      "type": "string"
    },
    "sns_topic_arn": {
      "value": "arn:aws:sns:eu-north-1:194442925473:umar-coldchain-excursions-terraform",
      "type": "string"
    },
    "sqs_dlq_url": {
      "value": "https://sqs.eu-north-1.amazonaws.com/194442925473/umar-coldchain-alert-dlq-terraform",
      "type": "string"
    },
    "sqs_queue_url": {

```

Terraform plan, apply and destroy = when ran it shows the (desired state) compared to the (actual state) and you can see what has changed before applying it. then destroy destroys everything so NEVER RUN IT only at the end of a project IF you don't need it anymore.

Terraform backend = files like the state file can be stored locally (on the working directory) or can be stored remotely on terraform's sight or company's can create their own. (AWS uses S3 buckets and DynamoDB) remotely better for big company's, collaboration with these key file and encryption possibilities etc.

There're two different beginning parts to a terraform block:

Resource = This I believe is creating a resource, then you put specific arguments and attributes to make the resources.

Data = The `terraform_data` managed resource type stores arbitrary data, triggers provisioners, and forces resource replacements without provisioning real cloud infrastructure. Also uses specific arguments and attributes.

NOTE AI: there are these on every resource that I have seen:

```
resource "aws_api_gateway_rest_api" "coldchain" {
```

Break it into two pieces:

```
1 | aws_api_gateway_rest_api . coldchain
2 |   ↑                       ↑
3 |   resource type          our name
```

The first part:

```
1 | aws_api_gateway_rest_api
```

Means:

AWS API Gateway REST API resource.

The second part:

```
1 | coldchain
```

is just the name **we gave it inside Terraform**.

For example:

```
1 | resource "aws_api_gateway_rest_api" "coldchain" {
```

means:

```
1 | Terraform resource name = coldchain
```

It does **not** mean AWS names it `coldchain`.

AWS sees:

```
1 | umar-coldchain-api-terraform
```

because of:

```
1 | name = var.api_name
```

```
resource "aws_api_gateway_rest_api" "coldchain" {
  name           = var.api_name
  description    = "Terraform cold-chain REST API"
  api_key_source = "HEADER"
}
```

So there are really two names:

```
1 | Terraform's internal name:
2 | coldchain
3 |
4 | Actual AWS name:
5 | umar-coldchain-api-terraform
```



LUKE EXPLANATION: there are these on every resource that I have seen:

```
resource "aws_api_gateway_resource" "branches" {
```

Luke just said that the whole thing is the name you gave it in terraform which is more believable because it does look like the part in a function and he also showed me why.



MY UNDERSTANDING: so I have been extremely confused on how the AI has made the resource/ how resources are even made in terraform. My current understanding shows that:

```
resource "aws_lambda_function" "link_dynamodb" {
```

So this is a resource and the first part in the "" is telling AWS what resource we are making in AWS. In this example we are making a lambda "aws_lambda_function"

Then the "link_dynamodb" is just the name we have given it in Terraform. we can call it in the terraform files with this name. (It is not its actual name).

AND

Everything is and can be connected differently for example I was looking at the API gateway configuration and to make new resources you can call the parent_id which is the original name of the API gateway, so if resource "aws_api_gateway_rest_api" "coldchain" then it would be aws_api_gateway_rest_api.coldchain.

Whereas Lambdas connect to other parts via

Then main thing that allows you too call other parts of terraform is the "aws_api_gateway_rest_api" "coldchain"

That is the **resource type**. Terraform knows what that means because you installed/configured the AWS provider in the [main.tf](#):

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
    }  
  }  
}
```

That tells Terraform: Use the HashiCorp AWS provider. The AWS provider contains definitions for loads of AWS resource types.

Variables and outputs

There are a few different types of variables:

- **Input variables:** (the one you have been using)

You have:

```
1 variable "api_name" {
2   description = "REST API Gateway name"
3   type       = string
4   default    = "umar-coldchain-api-terraform"
5 }
```

This does **not create an API Gateway**.

It is basically saying:

“Terraform, I want a variable called `api_name` . Unless I tell you otherwise, its value is `umar-coldchain-api-terraform` .”

The default is the name of the variable, that will be given when the “api_name” is inputted (Just like a normal variable)

- **Local variables:** (not too sure how these work: as LUKE)

When you reference these you use `local.name` but when we declare them/make them we use `locals` plural.

```
1 locals{
2   service_name = "Service Name"
3   owner        = "DevOps Directive"
4 }
```

These are basically temporary variables in a function, allows you to take a value that is repeated a few times through the code and pull it into a variable that can be re-used.

- **Output variables:** (not too sure how these work: as LUKE)

Using that same function analogy, these are the return values on a function. This allows us to take multiple terraform configuration and bundle them together.

The output variable might take the output of my apply and return an instance ip so that a different portion of my terraform conduction and consume that and uses it to set up configuration else where e.g. firewall

There are a number of ways too apply values to outputs: (least to most important)

1. manual entry during plan/apply

Terraform CLM will prompt you and ask you to input data, DON'T want to be doing this one because you can miss input data.

2. default value in declaration block (the one I am doing)

This is like a default blocking your output which it will uses if you don specify anything else.

3. TF_VAR_<name> environment variables

4. Terraform.tfvars file (this is the one the portal uses for work)

Used for the deploy script, and is really good for production environments.

3. *.auto.tfvars

4. Command line -var or -var-file

They can hold a variation of different values from: strings, booleans and numbers TO list(<TYPE>), set(<TYPE>), map(<TYPE>) and object({<ATTR NAME> = <TYPE>,...}) and validate is also done by terraform like type checking or you can write your own.

• The difference between variables and outputs

A variable is information going INTO Terraform **WHEREAS** An output is information coming OUT of Terraform

More advance language features

Function

- Collection
- Encoding
- Filesystem
- Date & Time
- Hash & Crypto
- IP Network

- Type Conversion

Expression

- Template strings
- Operations (!, -, *, /, %, >)
- Conditionals
- For ([for o in var.list : o.id])
- Splat (var.list[*].id)
- Dynamic blocks
- constraint (Types & versions)

Meta-arguments

Special arguments to control the behaviour of resources and/or modules. There are a few that the video calls out / mentions.

- **Depends on**

So normally there is a certain sequences of things that need to happen; e.g. your provisioning a server you need the ip address that is then passed on to a firewall use etc. when you run the terraform apply / plan command terraform will figure out the best path / sequence to take and how too take it for you.

However, there are some case where a resource implicitly relies/need to link to another one but there is no direct connection that terraform can take.

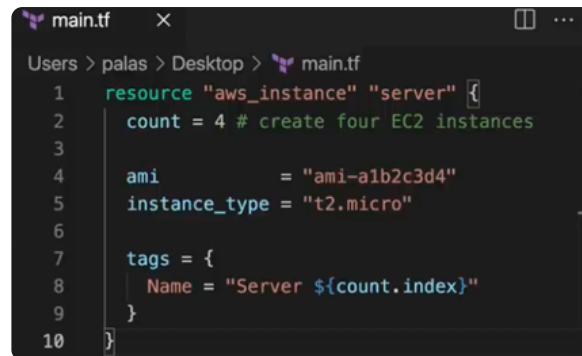
```
depends_on = [  
  aws_iam_role_policy_attachment.link_dynamodb_logs,  
  aws_iam_role_policy.link_dynamodb_access,  
  aws_cloudwatch_log_group.link_dynamodb  
]
```

The example above shows the lambda need access to the specific IAM rolls that I created and the cloud watch. And as there is no direct connection that terraform would automatically do I have used the “depends on” argument to basically tell terraform.

“Make sure to privations these policies before you provision the lambda because it depends on it”

- **Count**

Allows you to specify if you need multiple of the same configuration, lest say you have a block of code and you need to uses it multiple times by using count = 4 it will provision that resource/block of code 4 times. as you can see in the image bellow:

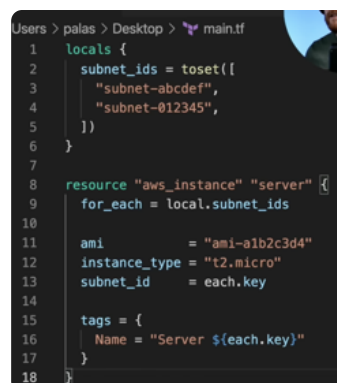


```
1 resource "aws_instance" "server" {
2   count = 4 # create four EC2 instances
3
4   ami           = "ami-a1b2c3d4"
5   instance_type = "t2.micro"
6
7   tags = {
8     Name = "Server ${count.index}"
9   }
10 }
```

The image also shows the tags having `${count.index}` which will Mae the names of the resource be 'name' 1, 'name' 2, 'name' 3 etc. it is very soul to uses this if you have multiple resources that are nearly identical.

• For_each

Kinda of like the count argument but gives you much more control over each resources. so we take an utterable to make the multiple reasoners. for example if you look at. the example below:



```
1 locals {
2   subnet_ids = toset([
3     "subnet-abcdef",
4     "subnet-012345",
5   ])
6 }
7
8 resource "aws_instance" "server" {
9   for_each = local.subnet_ids
10
11   ami           = "ami-a1b2c3d4"
12   instance_type = "t2.micro"
13   subnet_id     = each.key
14
15   tags = {
16     Name = "Server ${each.key}"
17   }
18 }
```

The subnets ids we have taken them and iterated each subnet id. It allows use to still create multiple but maintain the control to indivulise them as and when we need too.

 **NOTE:** `${.....}` this is a place holder for you to insert values in it for example:

Value = "Region is `${var.aws_region}`"

The var. is a variable with an output of eu-north-1

(JAVASCRIPT concept that terraform can uses)

 **NOTE:** AI: A usage plan is a set of rules that controls how fast and how often an API key can access specific application programming interfaces (APIs)

IF YOU WANT TO FINISH THE VIDEO ITS

=

Complete Terraform Course - From BEGINNER to PRO! (Learn Infrastructure as Code)
1:16:00

 **Questions / Queries / Pushback**

ASK LUKE = I thought yo shouldn't put sensitive data in the code base, because you said you put the secret keys in the Terraform.tfvars file. do you still have to like pass the passwords during run time or no??

=

The file is different for everyone, and it deploys for everyone's separate environment. and it is never pushed to git. git.ignored make sure that is always on